Windows OS

Day 1: Windows Basics & System Information

Files path : davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1

1.Write a script that prints the Windows version (e.g., Windows 10, 11) > 1.1.py

- I used Python platform module to detect OS and its release version

- It prints: Windows <release>

Screenshot:

```
[redacted]davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.1.py
Windows 10
```

2.Print the computer name and the current logged-in user > 1.2.py

- I used os.environ to access system environment variables

- COMPUTERNAME to display the machine name

- USERNAME to display the current logged-in user

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.2.py
Computer Name: DESKTOP[redacted]
Current User: B[redacted]
```

3.List all environment variables and their values > 1.3.py

- Used os.environ to access all system environment variables

- Looped through variables using .items()

- Printed each variable name and its value

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.3.py
ALLUSERSPROFILE = C:\ProgramData
APPDATA = C:\Users\[redacted]\AppData\Roaming
CHROME_CRASHPAD_PIPE_NAME = \\.\pipe\crashpad_4348_YWZJDXSVMUUWYDEG
COMMONPROGRAMFILES = C:\Program Files\Common Files
COMMONPROGRAMFILES(X86) = C:\Program Files (x86)\Common Files
COMMONPROGRAMW6432 = C:\Program Files\Common Files
COMPUTERNAME = DESKTOP-[redacted]
COMSPEC = C:\Windows\system32\cmd.exe
DRIVERDATA = C:\Windows\System32\Drivers\DriverData
```

4.Get and print the current working directory > 1.4.py

- Used os.getcwd() function to retrieve the current working directory

- The script prints the full path where the Python script is executed

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.4.py
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1
```

5.Find and display the system architecture (32-bit or 64-bit) > 1.5.py

- Used platform.architecture() to determine system architecture

- Retrieved the first element [0] from the returned tuple

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.5.py
64bit
```

6.Retrieve the current user's home directory path > 1.6.py

- Used os.path.expanduser("~") to get the current user's home directory

- The tilde symbol ~ represents the home folder

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.6.py
C:\Users\[redacted]
```

7.Write a script to detect if the system is Windows or not > 1.7.py

- Used platform.system() to detect the operating system

- Compared the result with the string "Windows"

- The script prints True if the system is Windows, otherwise False

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.7.py
True
```

8.Print the system boot time > 1.8.py

- I used venv to install and import psutil

- Used psutil.boot_time() to retrieve system boot timestamp

- Converted the timestamp to readable format using datetime.fromtimestamp()

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.8.py
2026-02-17 13:56:38
```

9.Retrieve and print the CPU name and number of cores > 1.9.py

- I used venv to install library

- Used py-cpuinfo library to get the official CPU name using get_cpu_info()["brand_raw"]

- Used get_cpu_info()["count"] to retrieve the number of CPU cores

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.9.py
CPU: Intel(R) Core(TM) i5-6440HQ CPU @ 2.60GHz
Cores: 4
```

10.Write a script that checks if a specific environment variable (like PATH) exists > 1.10.py

- Used os.environ to access system environment variables

- Checked if "PATH" exists inside environment dictionary

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 1>python3 1.10.py
PATH exists
```

Day 2: Processes, Tasks, and Services

Files path : davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2

`Note`: I am using Virtual Environment (venv) and all scripts run on it

1.List all running processes along with their PID (Process ID) > 2.1.py

- Used psutil.process_iter() to iterate through running processes

- Retrieved only pid and name fields

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.1.py
0 - System Idle Process
4 - System
108 - Registry
476 - smss.exe
504 - fontdrvhost.exe
528 - fontdrvhost.exe
704 - svchost.exe
712 - csrss.exe
724 - SearchProtocolHost.exe
736 - igfxEM.exe
800 - csrss.exe
```

2.Write a script that kills a process by its name > 2.1.py

- Used psutil.process_iter() to iterate through running processes

- Took process name from user input

- Compared it with running process names

- Used terminate() method to stop the matched process

Screenshot:

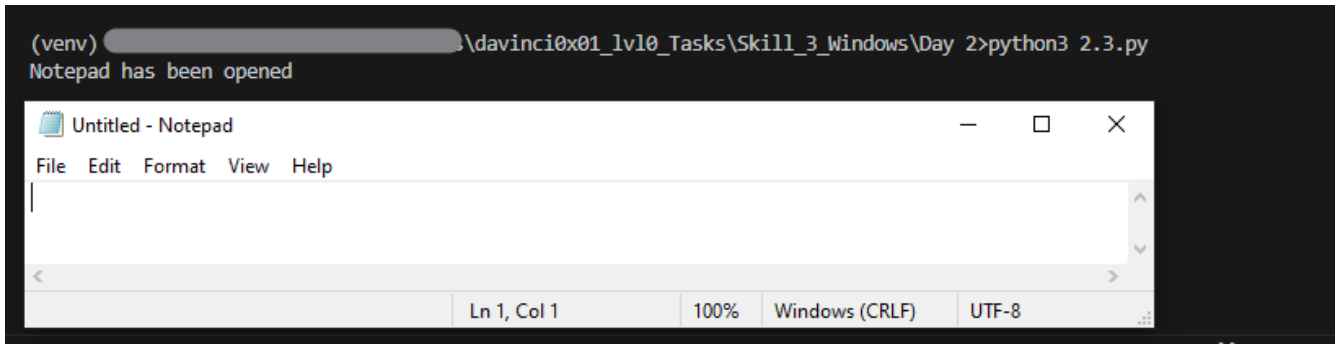
```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.2.py
Enter process name(EXP:notepad.exe): notepad.exe
Process terminated
```

3.Start a new process (like opening Notepad) from Python > 2.3.py

- Used subprocess.Popen() to start a new process

- Launched notepad.exe as an example

Screenshot:



4.Monitor and print CPU usage of the top 5 processes > 2.4.py

- Used psutil.process_iter() to loop through running processes

- Used cpu_percent(interval=0.1) to measure CPU usage

- Divided CPU usage by psutil.cpu_count() to normalize values (0–100%) like Task Manager

- Stored process name and CPU usage in a list

- Sorted processes in descending order using sorted() with reverse=True

- Displayed only the top 5 highest CPU-consuming processes.

- High "System Idle Process" means the CPU is mostly free.

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.4.py
System Idle Process - 76.375 %
brave.exe - 3.875 %
brave.exe - 3.875 %
MsMpEng.exe - 3.825 %
System - 0.0 %
```

5.Detect if a given process (like explorer.exe) is running > 2.5.py

- Used psutil.process_iter() to check running processes

- Used any() to determine if "explorer.exe" exists

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.5.py
Running
```

6.Write a script to restart a Windows service (like Spooler) > 2.6.py

- Used Windows command net stop and net start.

- Executed commands using subprocess.run()

- Administrator privileges are required

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.6.py
The Print Spooler service is stopping.
The Print Spooler service was stopped successfully.

The Print Spooler service is starting.
The Print Spooler service was started successfully.
```

7.List all services, their status (Running, Stopped), and startup type > 2.7.py

- Used psutil.win_service_iter() to retrieve Windows services.

- Converted service information using as_dict()

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.7.py
AJRouter - stopped - manual
ALG - stopped - manual
ApHidMonitorService - running - automatic
AppIDSvc - stopped - manual
Appinfo - running - manual
AppMgmt - stopped - manual
AppReadiness - stopped - manual
AppShellElevationService - stopped - manual
AppVClient - stopped - disabled
AppXSvc - stopped - manual
AssignedAccessManagerSvc - stopped - manual
```

8.Write a script to start, stop, or restart a service programmatically > 2.8.py

- Took service name and required action from user input

- Used Windows net command to start or stop services

- For restart, executed net stop then net start.

- Commands were executed using subprocess.run()

Screenshot:

```
[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.8.py
Service name: spooler
Action (start/stop/restart): stop
The Print Spooler service is stopping.
The Print Spooler service was stopped successfully.


[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 2>python3 2.8.py
Service name: spooler
Action (start/stop/restart): start
The Print Spooler service is starting.
The Print Spooler service was started successfully.
```

9.Create a simple task manager clone that refreshes every 5 seconds > 2.9.py

- Used psutil to retrieve running processes

- Calculated CPU usage and normalized it by dividing by CPU core count.

- Sorted processes by highest CPU usage and displayed top 5 processes

- Used while True with time.sleep(5) to refresh every 5 seconds

- Cleared screen using cls command

- Added try/except to handle NoSuchProcess and AccessDenied errors because some processes may terminate or be protected while scanning

Screenshot:

```
System Idle Process - 72.45 %
svchost.exe - 3.88 %
svchost.exe - 3.83 %
System - 0.0 %
Registry - 0.0 %
```

```
System Idle Process - 64.7 %
brave.exe - 7.78 %
System - 0.0 %
Registry - 0.0 %
smss.exe - 0.0 %
```

10.Find and terminate all processes started by a specific user > 2.10.py

- Took username and process name as input

- Iterated through running processes using psutil.process_iter()

- Checked both process owner and process name

- Terminated only matching processes

- Handled NoSuchProcess and AccessDenied exceptions

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows>python3 "Day 2\2.10.py"
Enter username: [redacted]
Enter process name (notepad.exe): notepad.exe
Terminated: notepad.exe
```

Day 3: File System & Automation

Files path : davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3

Note: I am using Virtual Environment (venv) and all scripts run on it

1.List all files and folders in C:\Users\Public > 3.1.py

- Used os.listdir() to retrieve all items inside C:\Users\Public

- Used raw string format r" " to handle Windows file path correctly

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.1.py
AccountPictures
Desktop
desktop.ini
Documents
Downloads
Libraries
Music
Pictures
Videos
```

2.Create a script that copies a file from one location to another > 3.2.py

- Used shutil.copy() to copy a file from source to destination

- Took source and destination paths from user input

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.2.py
Source file path: C:\Users\[redacted]\Desktop\test.txt
Destination path: C:\Users\[redacted]\Documents
File copied successfully
```

3.Write a Python tool that monitors a directory for file changes > 3.3.py

- Installed and used watchdog library to monitor filesystem changes

- Created a custom handler class that inherits from FileSystemEventHandler

- Implemented on_any_event() to catch all events (created/modified/deleted/moved)

- Printed event type and the path of the changed file/folder

- Used Observer to watch a target directory

- Set recursive=False to monitor only the selected folder (not subfolders).

- Kept the script running using an infinite loop with sleep(1) to reduce CPU usage.

- Stopped cleanly using CTRL + C (KeyboardInterrupt), then called observer.stop() and observer.join()

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.3.py
Enter folder path: C:\Users\[redacted]\Desktop
created -> C:\Users\[redacted]\Desktop\newfile.txt
modified -> C:\Users\[redacted]\Desktop\newfile.txt
deleted -> C:\Users\[redacted]\Desktop\newfile.txt
```

4.Find the size of every file inside a specific folder > 3.4.py

- Used os.walk() to traverse through a folder and its subdirectories.

- Constructed full file path using os.path.join()

- Used os.path.getsize() to retrieve file size in bytes

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.4.py
Enter folder path: C:\Users\[redacted]Desktop\test-path
test1.py - 22 bytes
test2.txt - 18 bytes
test3.txt - 0 bytes
```

5.Automatically create daily backups of a folder > 3.5.py

- Took source folder and backup destination from user input

- Used shutil.copytree() to copy the entire folder including all files and subfolders

- Added today's date to the backup folder name using date.today() to create unique daily backups

- The script can be scheduled to run automatically using Windows Task Scheduler to create daily backups without manual execution and the paths can be fixed directly inside the script instead of using input

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.5.py
Source folder: C:\Users\[redacted]\Desktop\test-folder
Backup base folder: C:\Users\[redacted]Desktop\
Backup created
```

6.Write a script that searches for a file by name across a directory tree > 3.6.py

- Used os.walk() to traverse through a folder and all subdirectories

- Checked if the target file name exists in the current folder

- Printed the full file path when found using os.path.join()

Screenshot:

```
(venv) [redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.6.py
Enter folder path: C:\Users\[redacted]\Desktop\test-folder
Enter file name: test3.txt
Found: C:\Users[redacted]\Desktop\test-folder\test-folder2\test3.txt
```

7.Monitor and report any new files created in C:\Windows\Temp > 3.7.py

- Used watchdog library to monitor C:\Windows\Temp

- Implemented on_created() to detect new files

- Printed the full path of newly created files

- Used Observer to continuously monitor the directory

- Script stops safely using CTRL + C

Screenshot:

```
(venv) C:\Users\      \Downloads\Codes\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.7.py
New file: C:\Windows\Temp\New Text Document.txt
New file: C:\Windows\Temp\New folder
```

8.Create a script that zips and unzips folders > 3.8.py

- Used built-in shutil module for compression and extraction

- For zipping: used shutil.make_archive() to create a .zip file from a folder

- For unzipping: used shutil.unpack_archive() to extract a zip file to a target folder

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.8.py
zip/unzip: zip
Folder path: C:\Users[redacted]\Desktop\test-folder
Output zip name (without .zip): zip-test
Zipped

(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.8.py
zip/unzip: unzip
Zip file path (.zip):[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3\zip-test.zip
Extract to folder:[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3\
Unzipped
```

9.List all files that were modified in the last 24 hours > 3.9.py

- Used os.walk() to traverse folder and subfolders

- Retrieved current time using time.time()

- Used os.path.getmtime() to get file modification time

- Compared difference to 86400 seconds (24 hours)

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.9.py
Enter folder path: C:\Users\[redacted]\Desktop\
Modified: C:\Users\[redacted]\Desktop\test-folder\test1.py
Modified: C:\Users\[redacted]\Desktop\test-folder\test2.txt
Modified: C:\Users\[redacted]\Desktop\test-folder\test-folder2\test3.txt
```

10.Write a tool that finds all files bigger than 100MB in a directory > 3.10.py

- Used os.walk() to traverse directories recursively

- Converted 100MB to bytes for accurate comparison

- Used os.path.getsize() to retrieve file size

Screenshot:

```
(venv)[redacted]\davinci0x01_lvl0_Tasks\Skill_3_Windows\Day 3>python3 3.10.py
Enter folder path:[redacted]\Downloads\Files
Large file:[redacted]\Downloads\Files\en_windows_7_ultimate_with_sp1_x64_dvd_u_677332.iso
Large file:[redacted]\Downloads\Files\kali-linux-2025.4-virtualbox-amd64.7z
Large file:[redacted]\Downloads\Files\ubuntu-18.04.5-live-server-amd64.iso
Large file:[redacted]\Downloads\Files\kali-linux-2025.4-virtualbox-amd64\kali-linux-2025.4-virtualbox-amd64.vdi
```